AI-Based Freelance-Startup System Progress Presentation

Week 02

Contents

CATALOGUE

01 Project Positioning

02 Weekly Progress

03 Planning and Next Steps

01

Project Positioning

Current Phase

Phase: Intelligent Foundation
Focus: System Design and Core Architecture
Goal of Week 02: Transform conceptual model into technical blueprint

Weekly Objective

01

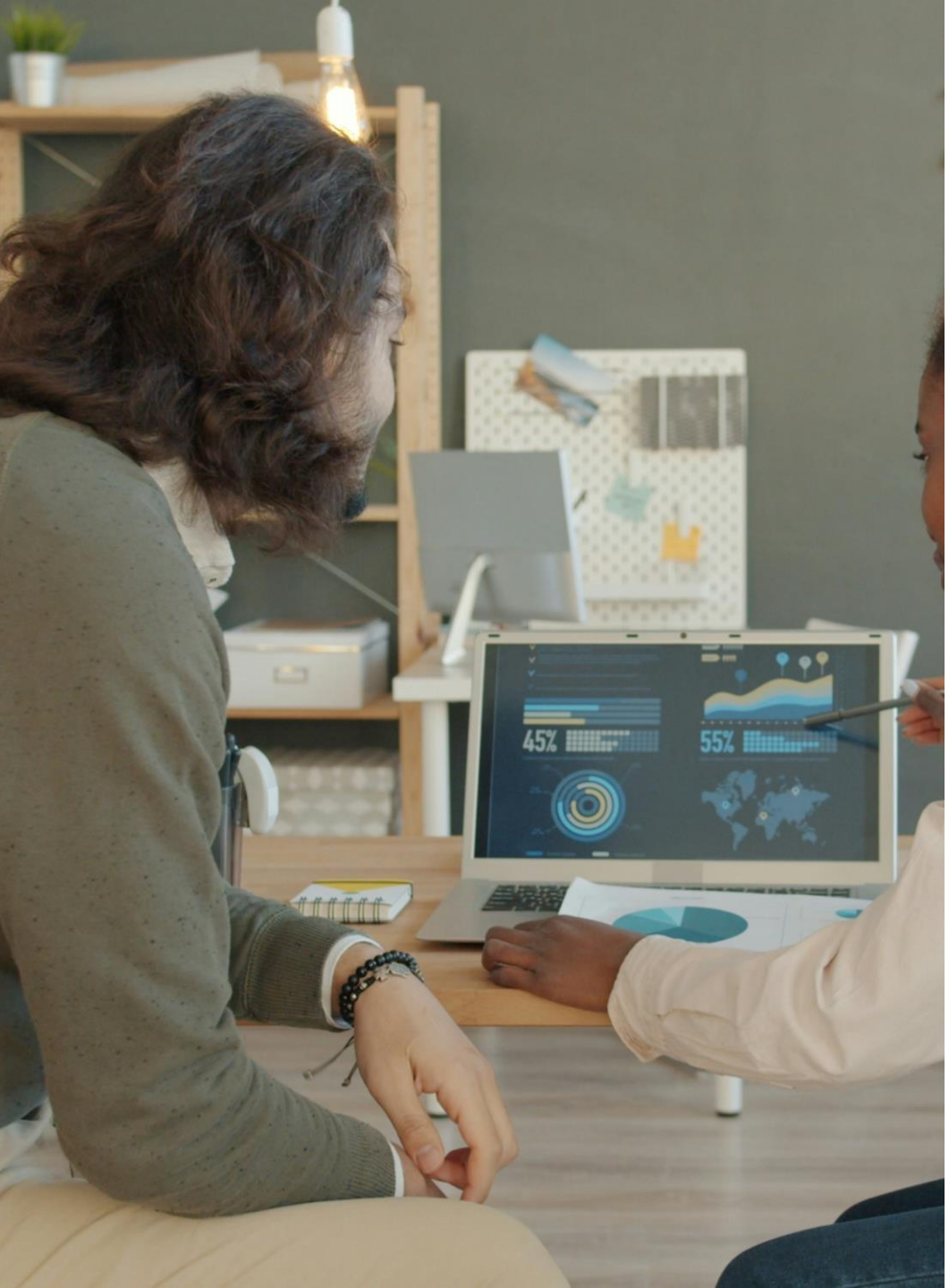
Design

Mockups

02

Database Conception

Entity Association Diagram
Databases Technologies



02

Weekly Progress

Work Completed This Week

Reasearch / Analysis tasks

Database Modeling
AI Orchestration Architecture
Back-end setup

Development tasks

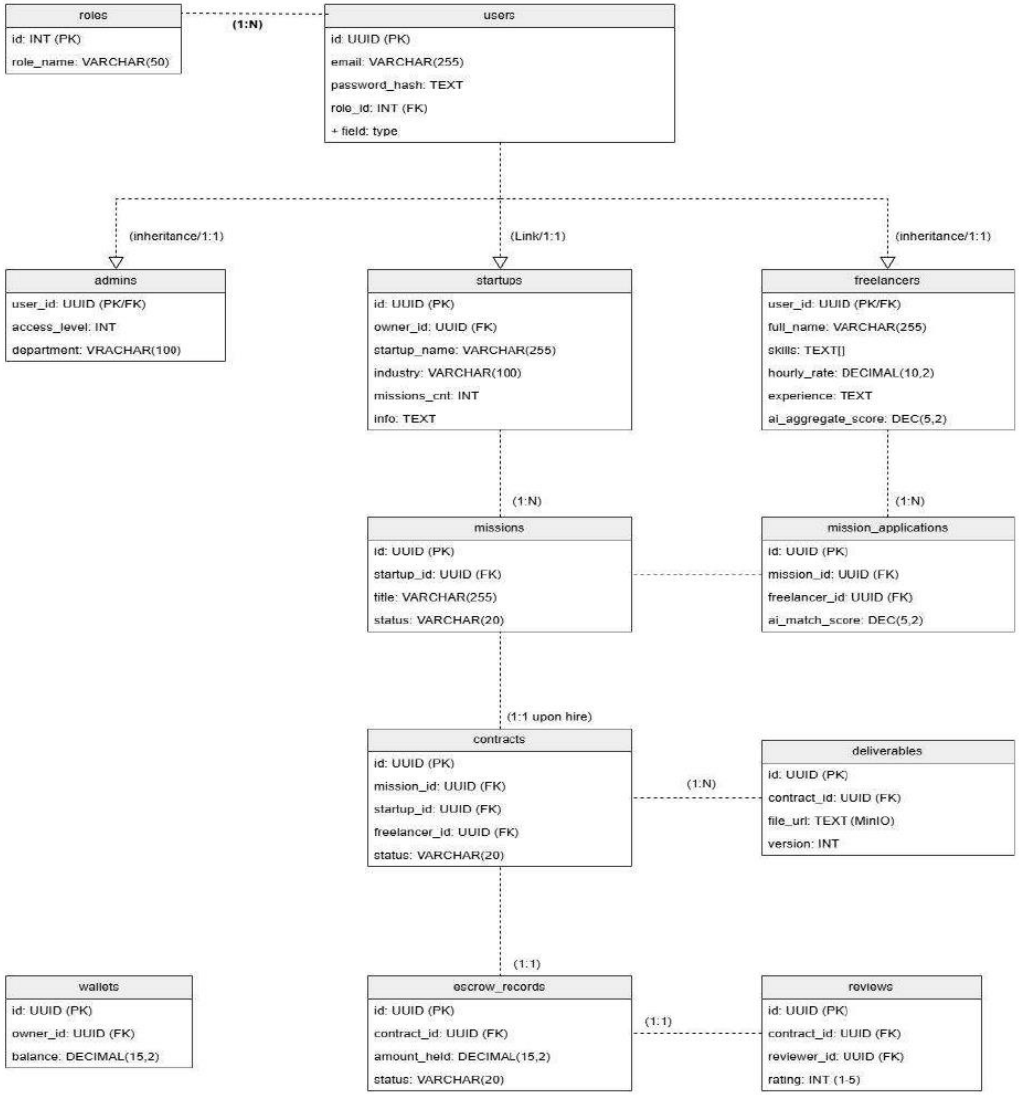
Front-end

Work Completed This Week

Reasearch / Analysis tasks

Database Modeling

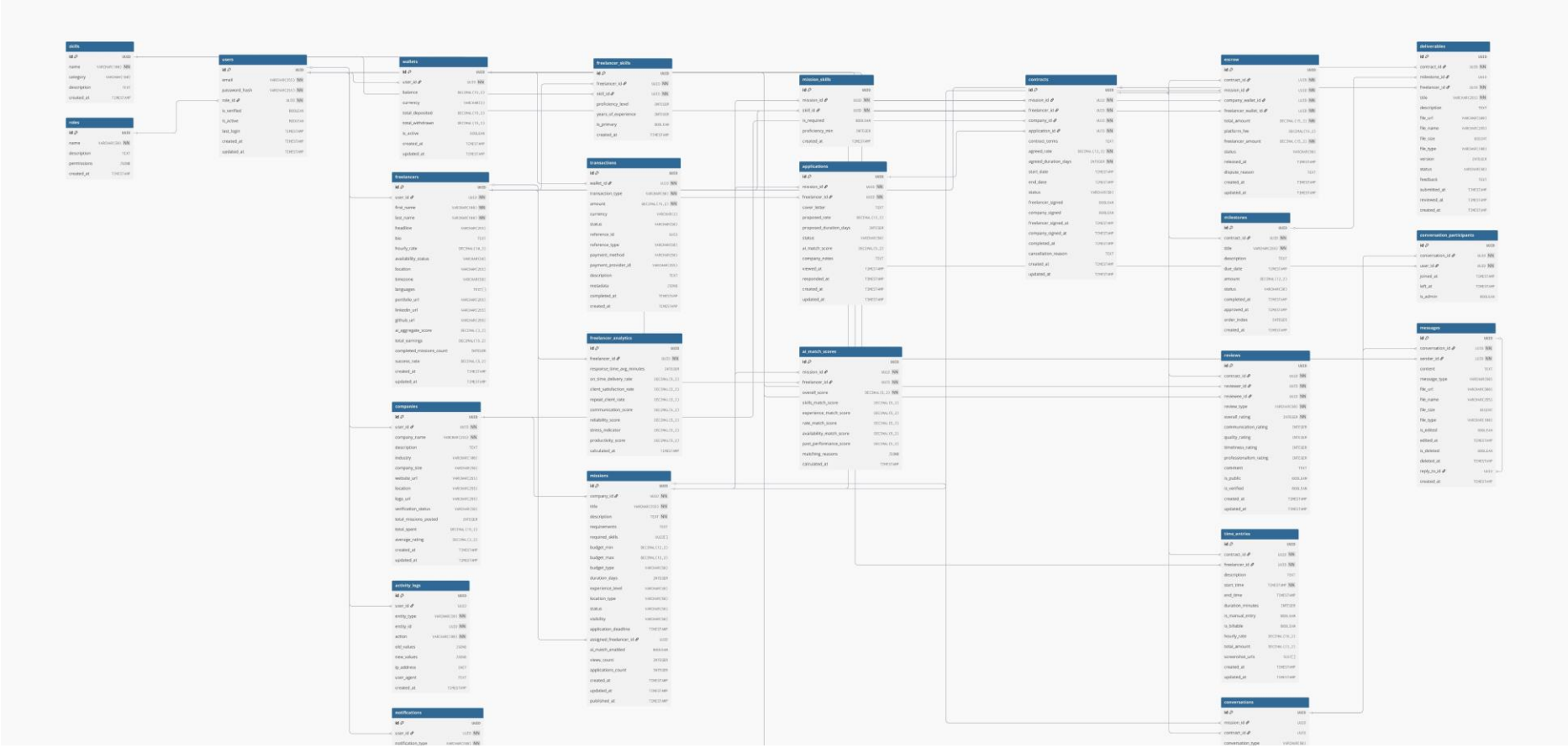
Started with small, service-specific tables, then integrated them into a **global schema** ensuring consistency, modularity, and alignment with the overall system architecture.



Work Completed This Week

Reasearch / Analysis tasks

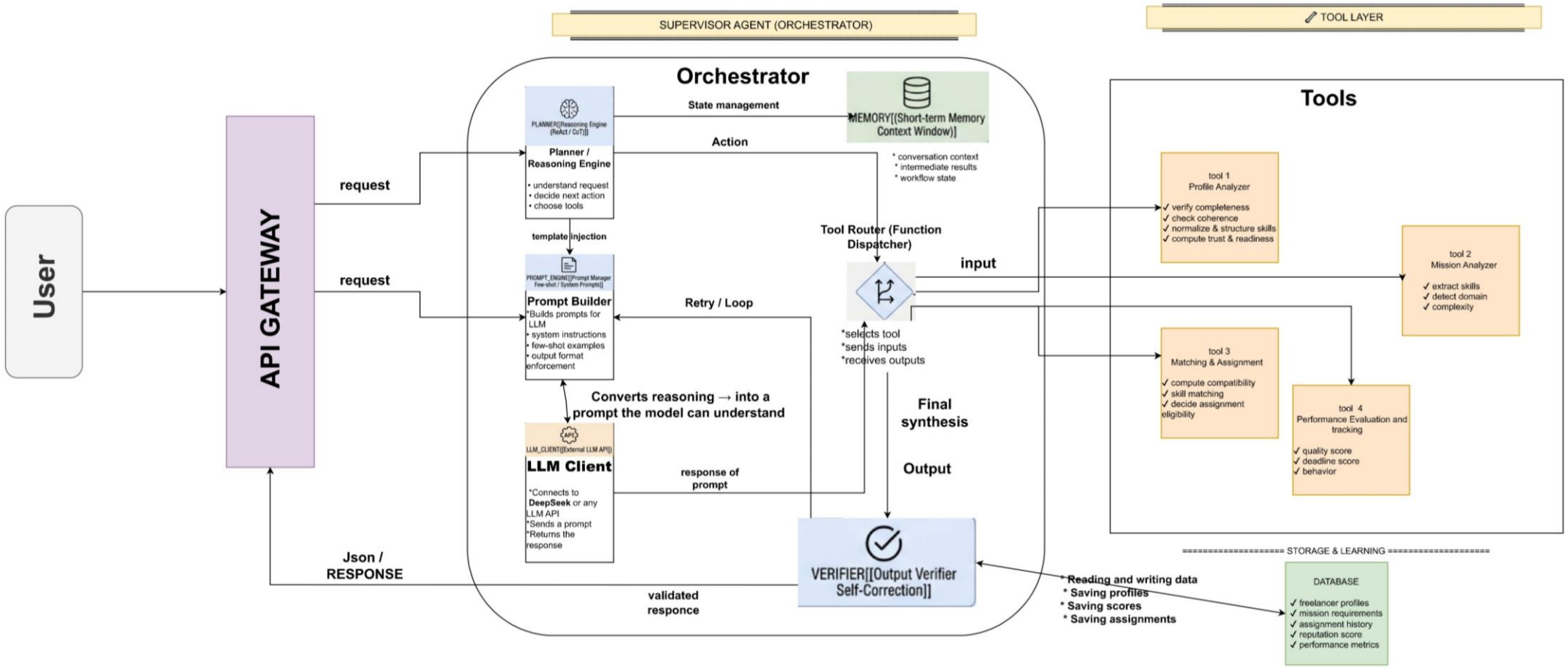
Database Modeling



Work Completed This Week

Reasearch / Analysis tasks

AI Orchestration Architecture



Work Completed This Week

Reasearch / Analysis tasks

AI Orchestration Architecture

USER **Utilisateur — Frontend**

Interface web · POST /api/ai/chat

L'utilisateur écrit un message depuis l'interface. La requête est envoyée à l'API avec un session_id unique qui identifie sa conversation, permettant à l'orchestrateur de retrouver l'historique.

ENVOIE

```
{
 "session_id": "abc-123",
 "message": "Find React dev 5+ years",
 "user_id": 42
}
```

REÇOIT

```
{
 "response": "J'ai trouvé 3 React devs...",
 "session_id": "abc-123",
 "tools_used": 2
}
```

BOUCLE INTERNE — WHILE TRUE · MAX 10 TOURS

L'orchestrateur tourne en boucle. À chaque tour il envoie le contexte à l'agent → l'agent décide → si tool_call : l'orchestrateur exécute et réinjecte le résultat → tour suivant. La boucle s'arrête quand l'agent retourne finish_reason : "stop".

Orchestrateur — envoie contexte complet à l'agent → **Agent décide** — tool_call ou texte final → **Tool exécuté** — Python → SQL → PostgreSQL → **Résultat injecté** — role:"tool" ajouté → rappelle l'agent

TOUR 1
Agent cherche les données
Pas de données encore → appelle search_freelancers
user "Meilleur React dev ?"
agent tool_call: search_freelancers
tool → [Yassine, Karim, Sara]

TOUR 2
Agent veut les scores
A la liste, mais pas les scores → appelle calculate_score ×3
agent tool_call: calculate_score(42)
tool → score: 87.4
agent tool_call: calculate_score(93)
tool → score: 91.0

TOUR 3 — STOP ✅
Agent a tout → texte final
finish_reason: "stop" → orchestrateur sort de la boucle et retourne la réponse
stop finish_reason: "stop"
agent "Sara est la meilleure (91/100)..."

↑ L'agent ne sait pas qu'il est dans une boucle — l'orchestrateur contrôle tout ↑

NODE B **AI Agent — Kimi K2.5 (LLM)**

Hugging Face Novita · Hébergé externement · Pas sur ton serveur

Le cerveau du système. Il reçoit le contexte complet (historique + message + descriptions des tools), réfléchit et prend une seule décision à chaque tour : soit appeler un tool pour obtenir des données, soit répondre directement en texte. Il ne contrôle pas la boucle — c'est l'orchestrateur qui décide de le rappeler.

A — Reçoit
Contexte complet
Historique + message actuel + TOOL_SCHEMAS (les descriptions JSON des tools disponibles)

B — Raisonne
Décide quoi faire
Ai-je assez de données pour répondre ?
Non → tool_call
Oui → texte final

C — Retourne
Répond à l'orchestrateur
"finish_reason": "tool_calls" ou "stop"

FastAPI
Endpoint /api/ai/chat. Reçoit, valide avec Pydantic, déclenche l'orchestrateur.

Orchestrateur
Chef d'orchestre. Tient memory{}, TOOL_SCHEMAS[],

AI Client
Wrapper InferenceClient HF. Envoie le payload à Kimi K2.5 chez Novita.

Tool Functions
3 fonctions Python (search, get_profile, calculate_score). Chacune → SQL → PostgreSQL.

Memory
Dict Python memory{session_id: [messages]} en dev. Redis en production.

PostgreSQL
Source de données des freelancers. Jamais appelée directement par l'agent.

Work Completed This Week

Reasearch / Analysis tasks

Back-end setup

> Docker Hub repository created
> GitHub repository initialized with defined structure
> Goal: Facilitate team development coordination and version control

Work Completed This Week

Development tasks

Front-end

- 80% completed
- Flexible to accommodate potential changes or adjustments

Current Challenges

Data Architecture Scalability Constraint

The system domains are too complex to be managed in a single database without creating tight coupling and scalability issues.

Microservices and Data Flow Clarity Gap

Need to clearly define service boundaries, database separation, Redis role, and AI integration before backend implementation.

AI Architecture Design Uncertainty

Difficulty choosing between a multi-agent architecture and a single-agent system with orchestration, impacting system complexity and maintainability.

Solutions and Decisions

Adoption of Polyglot Persistence Strategy

Separated storage by purpose:

- PostgreSQL (relational data)
- MongoDB (communication data)
- Redis (caching and streams)
- MinIO (object storage)

Standardized Collaboration and Deployment Workflow

Used GitHub for version control and Docker/Docker Hub for consistent environments and service containerization.

Single AI Agent with Central Orchestrator

Selected a centralized AI agent model to reduce complexity while keeping the architecture extensible.

Major Technical Decisions

Database Isolation per Microservice (Schema-Based)

Logical separation using SQL schemas to maintain modularity and ownership.

Redis as Event-Driven Backbone

Use Redis Streams to enable asynchronous communication and real-time status updates.

Architecture Blueprint Before Implementation

Completed detailed system design and data flow mapping prior to coding.

Week 1 Deliverables and Resources (Links)

Front-end

Back-end (Setup)

Database Modeling

AI Orchestration Architecture

03

Planning and Next steps

Plan for the Next Week

03

Develop Core Backend Features

implement selected core functionalities within the connected microservices to initiate real business logic execution.

02

Implement Schemas and Service Integration

implement the modular SQL schemas within PostgreSQL and connect each schema to its corresponding microservice.

01

Deploy PostgreSQL Infrastructure

deploy PostgreSQL using Docker to establish a stable, containerized relational database environment.

Thank you